Università degli Studi di Napoli Federico II

Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

---

Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato di Network Cloud Infrastructures

Prof. Giorgio Ventre

a.a. 2025-26

Studenti:

Francesco Fedele DE9000155

Giuseppe D'Angelo DE9000093

Sommario

Contents

Sommario **2**

1. Idea di base e obiettivo del progetto **3**

2. La topologia della rete **3**

3. Controller baseline **3**
3.1 Scenario 1: Assenza di attacchi 4
3.2 Scenario 2: Attacco Flood Costante 4
3.3 Scenario 3: Attacco a Burst 5
3.4 Confronto tra i tre scenari sul Controller Baseline 5

4. Controller aggiornato **5**
4.1 Fase di Warm-up e Calibrazione Iniziale 6
4.2 Scenario 1: Controller Aggiornato con Attacco Flood Costante 6
4.3 Scenario 2: Controller Aggiornato con Attacco a Burst 7
4.4 Scenario 3: Controller Aggiornato con Attacco Combinato (Costante + Burst) 7
4.5 Confronto tra i tre scenari sul Controller Aggiornato 8

1. Idea di base e obiettivo del progetto

In questo progetto abbiamo realizzato una rete SDN OpenFlow in grado di gestire attacchi di tipo Denial of Service (DoS), sfruttando il controller centralizzato SDN per eseguire tutti i controlli atti ad individuare eventuali anomalie. Esso infatti, essendo a conoscenza della topologia della rete e del traffico medio che la attraversa, può essere potenziato in modo da individuare gli attacchi semplicemente fornendogli opportuni strumenti di analisi.

Tramite questi ultimi, il controller diventa capace di riconoscere i flussi malevoli (anche qualora gli attaccanti dovessero cambiare porte TCP/UDP) e di bloccarli tramite una regola di drop che invia agli switch.

2. La topologia della rete

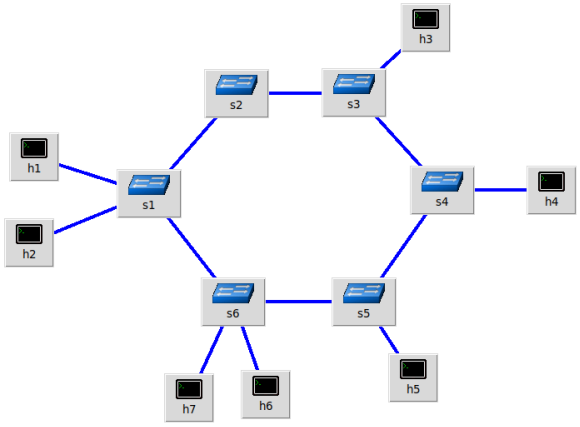


Figure 1: Topologia della rete

La rete in esame è costituita da:

- un controller SDN OpenFlow;
- sei switch OpenFlow collegati fra loro ad anello;
- sette host, di cui un server, due attaccanti e quattro legittimi.

I collegamenti fra gli switch sono tutti a 10 Mbps. All'avvio della rete, gli switch eseguono il protocollo STP. Si attende la convergenza di 35 secondi, al termine della quale sarà eletto s1 come bridge root e bloccato il collegamento fra s4 e s5.

Gli host legittimi comunicano tutti a 2Mbps. Di questi, h2, h4 e h6 usano TCP, mentre h7 usa UDP sulla porta 5004, ritenuta sicura.

L'host server h5 accetta connessioni UDP su porta 5001, TCP su 5002 e UDP safe su 5004.

Gli host attaccanti lanciano due tipi diversi di attacco:

- **h1, attacco flood costante**: genera un traffico costante e molto superiore al massimo consentito dalla rete (100 Mbps).
- **h3, attacco a burst**: ogni 4 secondi crea una nuova connessione UDP su cui invia tantissimi pacchetti in pochissimo tempo, arrivando a toccare i 100 Mbps.

3. Controller baseline

Nella prima parte dell'esperimento eseguiamo tre test della rete (320 s ciascuno, di cui 20 s iniziali di *warm-up* con i soli host TCP `h2, h4, h6`) con un controller di base che non gestisce gli attacchi.

3.1 Scenario 1: Assenza di attacchi

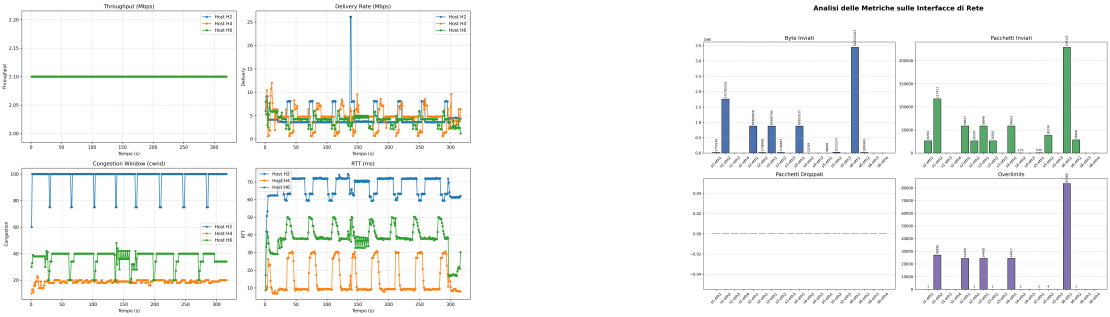


Figure 2: Baseline senza attacchi: metriche TCP degli host legittimi (sinistra) e statistiche delle porte degli switch (destra).

In assenza di attacchi (Figura 2), il carico massimo aggregato è ~ 8.4 Mbps < 10 Mbps e la rete opera in modo ideale:

- **Metriche Host:** Gli host TCP (`h2, h4, h6`) e l'host UDP sicuro `h7` (porta 5004) mantengono tutti un *throughput* costante di **2.10 Mbps** (80.1 MB per i TCP; 0% packet loss e 4.18 ms di jitter per `h7`). I valori di *cwnd* e *RTT* dipendono dal percorso: `h4` (su `s4`, 5 hop) ha RTT base ~ 10 ms (picchi a 30 ms) e cwnd $\approx$ 18–20; `h6` (su `s6`) ha RTT $\approx$ 38–50 ms e cwnd $\approx$ 38–40; `h2` (su `s1`) mantiene cwnd $= 100$ e RTT $\approx$ 60–72 ms.
- **Metriche Porte Switch:** Non si registrano pacchetti scartati. Il traffico cresce lungo l'albero STP verso `s5`: `s4-eth1`, `s3-eth1` e `s2-eth1` inoltrano ~ 87.9 MB (~ 58.6k pkt, flusso `h4`); `s1-eth2` inoltra 175.8 MB (117.4k pkt, `h4+h2`); `s6-eth1` aggrega tutti i flussi (`h4+h2+h6+h7`) con 344.5 MB (229.1k pkt).

3.2 Scenario 2: Attacco Flood Costante

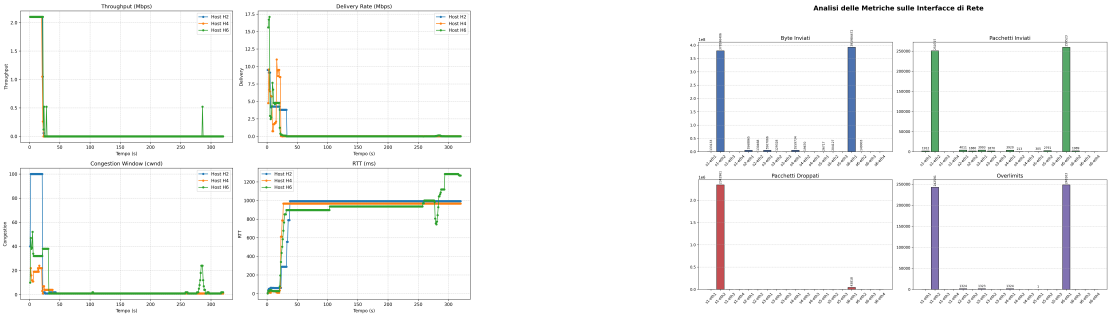


Figure 3: Baseline con flood costante da `h1`: metriche TCP degli host (sinistra) e statistiche delle porte degli switch (destra).

`H1` (su `s1`) inietta un flood UDP costante a **105 Mbps** (3.66 GB) verso `h5:5001` (Figura 3):

- **Metriche Host:** Appena parte l'attacco, i link a 10 Mbps `s1` $\rightarrow$ `s6` $\rightarrow$ `s5` vengono saturati. Per via delle perdite massicce, il controllo di congestione TCP di **`h2, h4, h6` azzera throughput e delivery rate** (≈ 0 Mbps, media finale 138–148 Kbps dovuta al solo warm-up), la *cwnd* crolla a 1–2 segmenti e l'*RTT* arriva a **900–1300 ms**. Anche il flusso UDP legittimo `h7` (su `s6`) crolla a **719 Kbps** medi con il **62% di packet loss** (30.004/48.463 persi) e jitter a 21.72 ms.
- **Metriche Porte Switch:** Come si vede dal grafico, la porta **`s1-eth2`** è quella che scarta più pacchetti (2.342.961 scartati, 242.591 overlimits, 378.6 MB inviati). Un secondo punto di drop è **`s6-eth1`** (43.818 scartati, 248.163 overlimits, 392.0 MB inviati), dove ai 10 Mbps in arrivo da `s1` si sommano `h6` e `h7`. Le porte a monte (`s4-eth1`, `s3-eth1`, `s2-eth1`) si fermano a 5.57 MB (~ 4.000 pkt), poiché `h4` rallenta a causa della congestione.

3.3 Scenario 3: Attacco a Burst

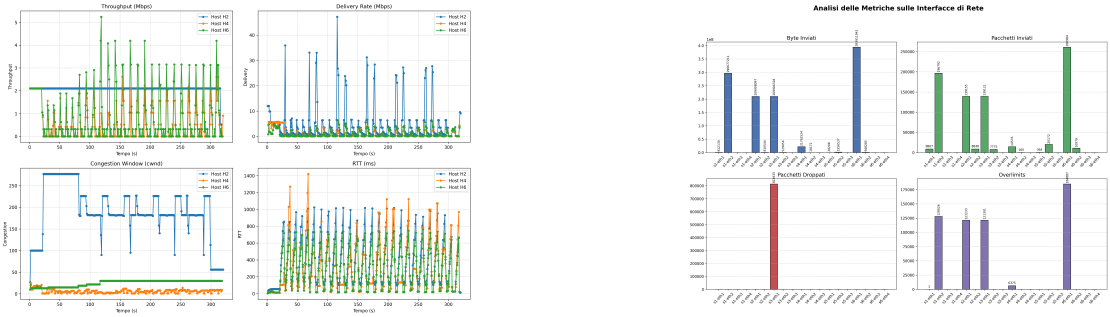


Figure 4: Baseline con attacco a burst da `h3`: metriche TCP degli host (sinistra) e statistiche delle porte degli switch (destra).

`H3` (su `s3`) alterna **burst UDP di 4 s a 105 Mbps** (50 MB a raffica cambiando porta sorgente) e pause di **4 s a** $\sim$ 500 **Kbps** (Figura 4):

- **Metriche Host: `H4` (su `s4`, a monte di `h3`)** è il più penalizzato, perché i suoi pacchetti attraversano direttamente la porta saturata `s3-eth1`: il throughput è azzerato ad ogni burst e il recupero nelle pause è solo parziale (**511 Kbps** medi, 19.6 MB), cwnd < 15 e picchi di RTT fino a **1200–1420 ms**. **`H6` (su `s6`, a valle)** presenta parametri che oscillano: durante le raffiche throughput si azzera e RTT arriva fino a 750 ms, mentre subito dopo il primo schizza a 3–5 Mbps e il secondo scende, a causa dello svuotamento delle code degli switch (**807 Kbps** medi, 30.9 MB inviati, cwnd $\approx$ 15–30). **`H2` (su `s1`)** riesce a mantenere il throughput più costante, con una media di **2.06 Mbps** (79.1 MB) e una cwnd elevata (180–280), ma presenta violente oscillazioni di RTT (60–1000 ms) e di delivery rate (25–47 Mbps). **`H7` (UDP su `s6`)** non subisce drop, ma vede la banda media ridotta a **1.64 Mbps** (cali periodici a 550 Kbps), jitter triplicato (**13.95 ms**) e 36 pacchetti *out-of-order*.
- **Metriche Porte Switch:** Nascendo l'attacco su `s3`, tutti i drop avvengono sul primo link a 10 Mbps, `s3-eth1` (**813.935 pacchetti droppati**, 121.301 overlimits, 209.9 MB inviati). A valle (`s2-eth1`, `s1-eth2` con 296.7 MB, `s6-eth1` con 393.9 MB), il flusso è già limitato a 10 Mbps: i drop sono quindi 0, ma gli *Overlimits* sono elevatissimi (fino a **184.887** su `s6-eth1`) per la saturazione ciclica delle code. Su `s4-eth1` il traffico scende a 21.8 MB per il rallentamento di `h4`.

3.4 Confronto tra i tre scenari sul Controller Baseline

La Tabella 1 riassume le differenze: mentre il **flood costante** (`h1`) causa un DoS totale sui flussi TCP e il 62% di loss sull'UDP saturando permanentemente `s1` $\rightarrow$ `s6` $\rightarrow$ `s5`, l'attacco **a burst** (`h3`) degrada pesantemente il throughput (`h4` -76%, `h6` -62%, `h7` -22%) e induce oscillazioni critiche di RTT (fino a 1.42 s) cambiando porta UDP ad ogni raffica.

Metrica / Osservazione	**No Attacchi**	**Flood Costante (`h1`)**	**Attacco a Burst (`h3`)**
Throughput medio TCP (`h2 / h4 / h6`)	2.10/2.10/2.10 Mbps	0.14/ $\approx$ 0/0.15 Mbps	2.06/0.51/0.81 Mbps
RTT massimo TCP (`h2 / h4 / h6`)	$\sim$ 72/30/50 ms	$\sim$ 1000/970/1300 ms	$\sim$ 1020/1420/750 ms
Flusso UDP `h7` (Banda / Loss / Jitter)	2.10 Mbps/0%/4.2 ms	0.72 Mbps/62%/21.7 ms	1.64 Mbps/0%/13.9 ms
Interfacce con Packet Drop	Nessuna (0 drop)	`s1-eth2` (2.34M), `s6-eth1` (43.8k)	`s3-eth1` (813.9k)

Table 1: Confronto sintetico delle prestazioni della rete con il controller baseline nei tre scenari di test.

4. Controller aggiornato

Nella seconda fase dell'esperimento testiamo la rete con il controller in grado di bloccare gli attacchi. Quest'ultimo è costituito da tre oggetti:

1. **`TrafficMonitor`:** ogni `MONITOR_INTERVAL` $= 2$ s invia richieste `OFPPortStatsRequest` a tutti gli switch e calcola il bitrate istantaneo $bps = 8 \cdot \Delta$rx_bytes$/\Delta t$ su ciascuna porta.
2. **`AnomalyDetector`:** mantiene per ogni porta uno storico costituito da un massimo di `WINDOW_SIZE` $= 10$ campioni di traffico lecito, con cui calcola una velocità threshold secondo la formula:

$$\text{Threshold} = \max\left(\mu + 2.5 \cdot \sigma,\ \texttt{ABSOLUTE_MIN_FLOOR_BPS} = 500\ \text{Kbps}\right)$$

dove μ è la media dei campioni memorizzati e σ la loro deviazione standard. In questo modo, il threshold si adatta alle condizioni di funzionamento della rete. Quando una porta è attraversata da un traffico che supera

il threshold, il controller la considera anomala e richiede le statistiche di tutti i flussi che la attraversano mediante `OFPFlowStatsRequest`. A questo punto verifica se esiste un flusso che contribuisce per almeno il 60% all'eccesso di banda. Se non lo trova interpreta l'anomalia come un aumento di traffico complessivo, quindi si limita ad aggiornare lo storico di campioni per la porta interessata. Se invece ne trova uno, controlla che non sia in **Whitelist** (`WHITELIST_IPS = {10.0.0.5}` e `WHITELIST_UDP_PORTS = {53, 123, 5004, 5201}`); se non è nemmeno in whitelist lo considera un flusso malevolo, quindi lo blocca.

3. **`MitigationEnforcer`:** esegue la procedura di blocco e sblocco dei flussi malevoli. Nello specifico, rimuove la regola di forwarding (priorità 1) e installa una regola di *drop* ad alta priorità (`PRIORITY_BLOCK` = 200). La durata del blocco cresce **esponenzialmente** in base alle recidive: (30 s → 60 s → 120 s → 240 s). Inoltre, traccia in `seen_ports` le porte L4 sorgenti usate da ciascun host: se uno stesso IP attacca da `MAX_PORT_ATTACKS` = 3 o più porte distinte, il controller scala automaticamente a un **blocco globale L3 sull'IP sorgente**.

4.1 Fase di Warm-up e Calibrazione Iniziale

All'avvio dell'esperimento, vengono attivati per i primi 20 s i soli flussi TCP legittimi (`h2, h4, h6`) per consentire al controller di calibrare le soglie prima dell'ingresso di `h7` e degli attaccanti:

```
*** Avvio server iperf su h5 (UDP attacco:5001, TCP legit:5002, UDP legit:5004)
*** Avvio traffico legittimo iniziale (h2, h4, h6 -> h5 TCP per 320s)
*** Attesa calibrazione baseline controller sul traffico lecito (20s)...
*** Avvio flusso UDP legittimo (h7 -> h5:5004 per 300s)
*** Avvio attacco (h1 flood costante 100M su UDP:5001)
*** Esperimento in corso per 300s
```

Parallelamente, il log di avvio del controller mostra gli `WARMUP_CYCLES` = 8 cicli di campionamento (da 2 s ciascuno) al termine dei quali entra in funzione l'enforcement:

```
ryu-manager controller.py
instantiating app controller.py of DoSController
[WARMUP] Calibrazione baseline in corso... (7 cicli rimanenti)
...
[WARMUP] Calibrazione baseline in corso... (0 cicli rimanenti)
[WARMUP] Fase di warm-up completata! Sistema a regime (Enforcement attivo).
```

4.2 Scenario 1: Controller Aggiornato con Attacco Flood Costante

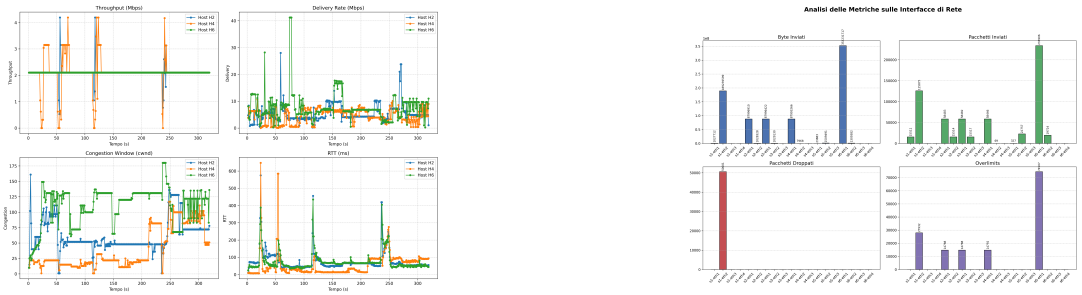


Figure 5: Controller aggiornato con flood costante (`h1`): metriche host (sinistra) e statistiche porte switch (destra).

Nel primo test (Figura 5), `H1` (`10.0.0.1`) avvia il flood costante a 105 Mbps da porta UDP 48710 verso `h5:5001`. Dal log del controller si osserva la sequenza di mitigazione con blocco esponenziale:

```
[MITIGATION: 14:56:47] Blocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710, 'udp_dst': 5001} su switch 1 per
      30s (recidiva #1, porte_distinte=1)
[MITIGATION: 14:56:49] Blocco flow {'in_port': 2, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710, 'udp_dst': 5001} su switch 5 e
      switch 6 per 30s (recidiva #1, porte_distinte=1)
[MITIGATION: 14:57:17] Sblocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710} su switch 1
[MITIGATION: 14:57:19] Blocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710, 'udp_dst': 5001} su switch 1 per
      60s (recidiva #2, porte_distinte=1)
[MITIGATION: 14:58:19] Sblocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710} su switch 1
[MITIGATION: 14:58:21] Blocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.1', 'udp_src': 48710, 'udp_dst': 5001} su switch 1 per
      120s (recidiva #3, porte_distinte=1)
```

- **Metriche Host:** A differenza di ciò che accade con il controller baseline, qui **`h2, h4, h6` mantengono tutti una media di 2.10 Mbps** (80.1–80.2 MB trasferiti). Nei grafici di Throughput e RTT si notano solo **4 brevi transitori** ($T \approx 22\,\text{s}, 52\,\text{s}, 115\,\text{s}, 238\,\text{s}$), corrispondenti esattamente agli istanti in cui, allo scadere di ciascun cooldown (30 s → 60 s → 120 s → 240 s), il controller sblocca, rileva e blocca nuovamente l'attacco. Subito dopo ogni blocco, `h2` e `h4` mostrano un picco di recupero a 3.1–4.2 Mbps e l'RTT torna stabile sotto i 100 ms, mentre `h6` resta fisso a 2.10 Mbps. Il flusso UDP `h7` (riconosciuto in whitelist su porta 5004) ottiene

2.09 Mbps medi con **0% di packet loss** (contro il 62% ottenuto con il controller di base) e jitter di soli 2.33 ms.

- **Metriche Porte Switch:** I pacchetti scartati su **`s1-eth2`** scendono da 2.342.961 a soli **50.601** (−97.8%, dovuti unicamente al tempo che intercorre fra lo sblocco di h1 e il successivo nuovo blocco), mentre su **`s6-eth1` i drop scendono a 0** (da 43.818). Le porte `s4-eth1`, `s3-eth1` e `s2-eth1` tornano a inoltrare gli 88.0 MB completi di `h4`, e gli *Overlimits* su `s6-eth1` scendono a 74.507 (valore analogo al caso senza attacchi).

4.3 Scenario 2: Controller Aggiornato con Attacco a Burst

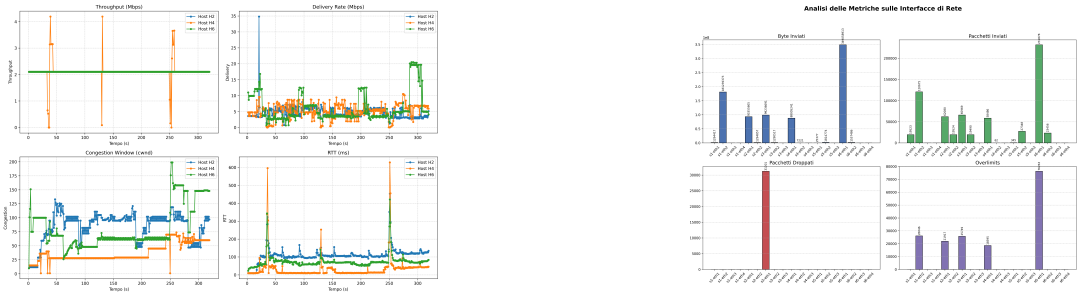


Figure 6: Controller aggiornato con attacco a burst (`h3`): metriche host (sinistra) e statistiche porte switch (destra).

In questo scenario (Figura 6), `h3` (`10.0.0.3`) invia burst a 105 Mbps cambiando porta UDP ogni 4 s. Da notare che quando il controller blocca tre o più flussi con stesso IP sorgente ma porte UDP diverse, **inizia a generare regole di drop solo su tale indirizzo IP**. In questo modo, qualsiasi nuovo flusso che parte da quell'IP viene subito contrassegnato come malevolo e bloccato.

```
[MITIGATION: 15:8:47] Blocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.3', 'udp_src': 48008, 'udp_dst': 5001} su switch 3 per
    30s (recidiva #1, porte_distinte=1)
[MITIGATION: 15:8:53] Blocco flow {'in_port': 3, ..., 'ipv4_src': '10.0.0.3', 'udp_src': 36197, 'udp_dst': 5001} su switch 3 per
    30s (recidiva #1, porte_distinte=2)
[MITIGATION: 15:9:1]  Blocco flow {'eth_type': 2048, 'ipv4_src': '10.0.0.3', 'in_port': 3} su switch 3 per 30s (recidiva #1,
    porte_distinte=3)
[MITIGATION: 15:9:31] Sblocco flow {'eth_type': 2048, 'ipv4_src': '10.0.0.3', 'in_port': 3} su switch 3
[MITIGATION: 15:9:33] Blocco flow {'eth_type': 2048, 'ipv4_src': '10.0.0.3', 'in_port': 1/2} su switch 1, 2, 6 per 30s (recidiva
    #1, porte_distinte=3)
```

- **Metriche Host:** Mentre con il controller di base `h4` e `h6` crollavano a 0.51 e 0.81 Mbps con oscillazioni continue, qui **tutti gli host TCP (`h2`, `h4`, `h6`) raggiungono 2.10 Mbps medi** (80.1–80.2 MB). `h2` e `h6` mantengono un throughput piatto a 2.10 Mbps per tutti i 320 s, mentre `h4` mostra solo 3 brevi avvallamenti ($T \approx 35\,\text{s}, 130\,\text{s}, 250\,\text{s}$, in corrispondenza delle scadenze dei blocchi prima del raddoppio del backoff) recuperati istantaneamente con picchi a 3.6–4.2 Mbps. L'RTT fuori dai 3 transitori resta bassissimo (~ 15 ms per `h4`, ~ 70 ms per `h6`, ~ 100 ms per `h2`). Il flusso UDP `h7` recupera pienamente a **2.09 Mbps** (0% loss, jitter ridotto da 13.95 ms a 5.48 ms).
- **Metriche Porte Switch:** Grazie al blocco per IP su `s3`, i pacchetti scartati su **`s3-eth1`** crollano da 813.935 a soli **31.201** (−96.2%). Il traffico di `h4` su `s4-eth1` risale da 21.8 MB a 88.0 MB e gli *Overlimits* su `s6-eth1` si dimezzano da 184.887 a 76.263.

4.4 Scenario 3: Controller Aggiornato con Attacco Combinato (Costante + Burst)

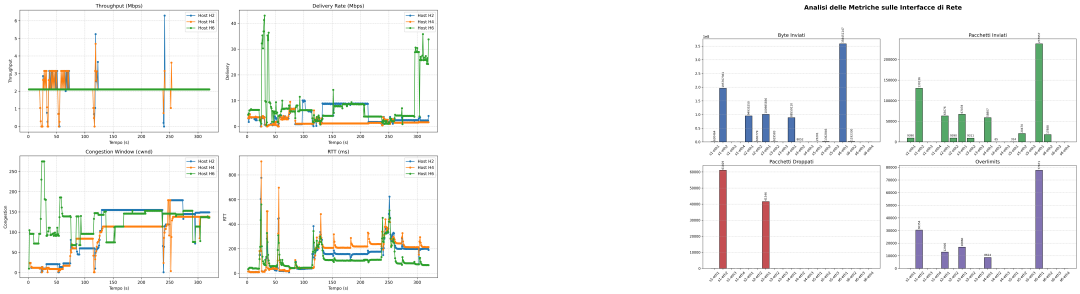


Figure 7: Controller aggiornato con doppio attacco simultaneo (`h1` costante + `h3` burst): metriche host (sinistra) e switch (destra).

Nell'ultimo test (Figura 7), entrambi gli attaccanti agiscono simultaneamente (`h1` flood costante su `s1` e `h3` burst su `s3`). Il controller combina il blocco esponenziale sul flusso di `h1` e l'escalation al blocco L3 sull'IP di `h3` dopo 3

porte distinte:

- **Metriche Host:** Nonostante il doppio attacco da 210 Mbps complessivi, **`h2, h4, h6` riescono a mantenere il proprio throughput a una media di 2.10 Mbps** (80.1–80.2 MB). Dopo un breve transitorio iniziale tra $T = 20$ s e $T \approx 65$ s (dovuto all'individuazione del flusso partito da `h1` e di quelli da `h3`) e due brevi finestre di ri-blocco a $T \approx 118$ s e $T \approx 245$ s (compensate da picchi TCP fino a 5–6 Mbps), il throughput rimane fisso a 2.10 Mbps con RTT contenuto (≤ 200 ms a regime). L'host UDP `h7` mantiene **2.07 Mbps** medi (74.0 MB) con **0% di packet loss** e 4.85 ms di jitter.
- **Metriche Porte Switch:** I drop restano confinati ai soli istanti di rilevamento sulle due porte di ingresso degli attacchi: **61.029 drop su `s1-eth2`** (per `h1`) e **41.546 drop su `s3-eth1`** (per `h3`), con **0 drop su `s6-eth1`** e 77.671 overlimits, garantendo la piena operatività della rete.

4.5 Confronto tra i tre scenari sul Controller Aggiornato

La Tabella 2 riassume i risultati ottenuti con il controller aggiornato utilizzando le stesse metriche della Tabella 1: in tutti e tre gli scenari d'attacco il throughput dei flussi TCP legittimi (`h2, h4, h6`) viene ripristinato a 2.10 Mbps, la perdita sul flusso UDP `h7` si azzera (0%) e i drop sulle porte degli switch vengono abbattuti di oltre il 96%.

Metrica / Osservazione	**Flood Costante (`h1`)**	**Attacco a Burst (`h3`)**	**Combinato (`h1 + h3`)**
Throughput medio TCP (`h2 / h4 / h6`)	2.10/2.10/2.10 Mbps	2.10/2.10/2.10 Mbps	2.10/2.10/2.10 Mbps
RTT massimo TCP (`h2 / h4 / h6`)	$\sim$ 575/647/435 ms	$\sim$ 453/628/421 ms	$\sim$ 777/910/560 ms
Flusso UDP `h7` (Banda / Loss / Jitter)	2.09 Mbps/0%/2.3 ms	2.09 Mbps/0%/5.5 ms	2.07 Mbps/0%/4.8 ms
Interfacce con Packet Drop	`s1-eth2` (50.6k, -97.8%)	`s3-eth1` (31.2k, -96.2%)	`s1-eth2` (61.0k), `s3-eth1` (41.5k)

Table 2: Confronto sintetico delle prestazioni della rete con il controller aggiornato nei tre scenari di test.